Les tests unitaires avec les mocks

Tous droits réservés à STUDI - Reproduction interdite

Table des matières

I. Contexte **3**

II. Tests unitaires et mocking **3**

A. Tests unitaires et mocking 3

B. Exercice : Quiz 7

III. Approfondissement des doubles de test avec Symfony : Spies et Mocks **8**

A. Approfondissement des doubles de test avec Symfony : Spies et Mocks 8

B. Exercice : Quiz 11

IV. Essentiel **12**

V. Auto évaluation **12**

A. Exercice 12

B. Test 13

Solutions des exercices **14**

Tous droits réservés à STUDI - Reproduction interdite

I. Contexte

Durée : 1 heure.

Environnement de travail : PC ou Mac avec 8 Go de RAM minimum.

Prérequis : notion PHP, Symfony.

Contexte

Les tests unitaires garantissent non seulement que votre code fonctionne comme prévu, mais ils peuvent également faciliter la maintenance et la mise à jour de votre code à long terme. Et pourtant, l'écriture de tests unitaires peut parfois être une tâche complexe, surtout lorsqu'elle implique des composants ou des services externes.

Imaginez que vous construisez une application météorologique qui utilise un service web pour récupérer les données de température. Écrire des tests unitaires pour ce service peut être un défi : vous ne pouvez pas simplement « *éteindre* » le temps réel pour vos tests. Ou encore, imaginez que vous développiez une application de e-commerce qui interagit avec un service de paiement en ligne. Vous ne voudriez certainement pas effectuer de véritables transactions chaque fois que vous exécutez vos tests.

C'est ici que les mocks peuvent être votre meilleur allié. Les mocks vous permettent de simuler ces services externes, de sorte que vous pouvez vous concentrer sur le test de votre propre code, tout en ayant le contrôle total sur le comportement des dépendances externes.

Intéressé par les tests unitaires avec les mocks ? Ce cours vous guidera à travers les concepts clés, vous montrera comment utiliser PHPUnit pour créer et utiliser des mocks, et vous donnera des exemples concrets pour vous aider à comprendre comment les mocks peuvent améliorer la qualité et la fiabilité de vos tests unitaires.

II. Tests unitaires et mocking

A. Tests unitaires et mocking

Définition **Tests unitaires**

Les tests unitaires sont des évaluations ciblées de petits segments de code, comme les fonctions ou les méthodes. Le but est de vérifier que chaque élément fonctionne correctement de manière isolée. Les tests unitaires facilitent également la maintenance et l'évolution du code, car ils permettent de vérifier rapidement que les modifications apportées n'ont pas introduit de nouveaux bugs.

Lorsque le code à tester interagit avec des ressources externes (comme des bases de données ou des appels réseau), ces tests deviennent dépendants de ces ressources, ce qui peut les ralentir et les rendre moins fiables. Pour pallier cela, nous utilisons des « *mocks* ». Un mock est une simulation programmée d'un objet réel qui reproduit son comportement de manière contrôlée. Les mocks permettent d'isoler le code à tester et de maîtriser le comportement des dépendances, facilitant ainsi le test de tous les scénarios possibles de manière rapide et indépendante.

Concepts fondamentaux de Mocking

Le *mocking* se réfère à la pratique de créer des objets d'imitation, appelés « *mocks* », qui reproduisent le comportement des objets réels dans un système. Lorsqu'une portion de votre code nécessite une interaction avec une base de données, vous pouvez créer un « *mock* » de cette base de données pour les besoins de vos tests. Au lieu d'un véritable appel à la base de données, qui peut être lent et imprévisible, votre mock de base de données se comportera exactement comme vous le définissez.

On fait appel aux mocks lorsqu'on veut isoler une partie du code pour la tester indépendamment de ses interactions avec d'autres éléments du système. C'est particulièrement utile lorsque ces interactions sont complexes, lentes, ou imprévisibles.

L'utilisation des mocks offre des avantages non négligeables. Cependant, il faut en faire usage avec discernement pour ne pas complexifier outre mesure les tests, ou obtenir des résultats qui ne traduisent pas fidèlement le comportement réel du système. Les mocks sont certes des outils précieux dans la boîte à outils de tout développeur de tests, mais ils doivent être utilisés judicieusement.

En dehors des bases de données et des appels réseau, il existe d'autres types de dépendances externes qui peuvent être isolées par des mocks. Quelques exemples :

- Système de fichiers : lorsque votre code interagit avec le système de fichiers (comme la lecture ou l'écriture de fichiers), vous pouvez créer un mock pour simuler ces interactions. Cela vous permet de tester comment votre code gère différents scénarios, comme l'absence d'un fichier ou un espace disque insuffisant.
- Horloge système : si votre code dépend de l'heure ou de la date actuelle, un mock de l'horloge système peut vous aider à tester comment votre code gère les dates et les heures. Ceci est particulièrement utile si votre logique repose sur des délais ou des échéances.

Dans le cadre d'une utilisation avec Symfony, le composant Filesystem est souvent utilisé pour les interactions avec le système de fichiers. Vous pouvez créer un mock de Filesystem pour isoler ces interactions dans vos tests. Quant à l'horloge système, bien que Symfony n'offre pas de composant intégré pour cela, vous pouvez utiliser le paquet lcobucci/clock, qui fournit une interface Clock que vous pouvez mocker.

Complément **Limitations du mocking**

Bien que le mocking soit une technique très puissante, il est important de l'utiliser avec discernement. En effet, l'utilisation excessive de mocks peut conduire à des tests qui ne reflètent pas le comportement réel du système. Par exemple, si vous mockez une dépendance qui a un bug, vos tests ne révéleront pas ce bug puisque le mock ne l'aura pas.

De plus, les mocks peuvent conduire à des tests qui sont très couplés à l'implémentation spécifique du code. Cela signifie que si vous changez la façon dont votre code est implémenté (mais pas son comportement), vous pourriez avoir besoin de réécrire vos tests même si le comportement attendu du système n'a pas changé. Par exemple, si vous mockez une méthode qui est ensuite refactoring, vous devrez mettre à jour votre mock pour refléter cette modification.

Enfin, les mocks peuvent rendre les tests plus complexes et plus difficiles à comprendre. C'est particulièrement le cas si vous utilisez beaucoup de mocks, ou des mocks qui ont des comportements très élaborés. C'est pourquoi il est souvent recommandé de n'utiliser des mocks que pour isoler le code que vous testez des aspects du système qui sont lents, non déterministes ou difficiles à contrôler (comme les bases de données, les appels réseau, le système de fichiers, l'horloge système, etc.).

Préparation de l'environnement de test

Avant de commencer les tests unitaires, il est crucial de préparer et de configurer correctement l'environnement de test. Cela peut impliquer un certain nombre de tâches préparatoires :

1. La définition de variables d'environnement spécifiques aux tests : par exemple, si votre application utilise des variables d'environnement pour configurer l'accès à une base de données, vous voudrez probablement définir ces variables différemment pour les tests, peut-être pour pointer vers une base de données de test distincte.
2. L'initialisation d'une base de données de test : si votre application interagit avec une base de données, vous devrez peut-être initialiser une base de données de test avec un état connu avant chaque test. Cela peut être accompli en utilisant des migrations de base de données, ou en insérant manuellement des données nécessaires avant chaque test.
3. La création de dummies ou de stubs pour les ressources externes : si votre application dépend de services externes, vous devrez peut-être créer des doubles de test pour ces services, pour éviter que vos tests n'aient à faire de véritables appels réseau.

Tous droits réservés à STUDI - Reproduction interdite

Méthode

Dans le contexte de Symfony et PHPUnit, vous pouvez effectuer ces tâches préparatoires dans la méthode setUp() de votre classe de test. PHPUnit appelle automatiquement cette méthode avant chaque test, donc tout code que vous y placez sera exécuté avant chaque test.

Lors de l'utilisation de PHPUnit dans Symfony, les variables d'environnement définies dans le fichier .env.test seront lues par défaut. Cela signifie que si vous avez une variable DATABASE_URL définie dans ce fichier, il n'est pas nécessaire d'utiliser la fonction `putenv()` pour la définir à nouveau.

Cependant, si vous avez besoin de définir des variables d'environnement spécifiques pour vos tests ou d'initialiser une base de données de test, vous pouvez le faire dans la méthode `setUp()`.

Exemple

```php
namespace Tests\AppBundle;

use PHPUnit\Framework\TestCase;

class MonTest extends TestCase
{
   protected function setUp(): void
   {
        // Définir une variable d'environnement
putenv('DATABASE_URL=mysql://testuser:testpass@localhost/testdb');

        // Initialiser la base de données de test
        // Ceci est juste un exemple, la méthode réelle dépendra de votre système de gestion de
base de données
        shell_exec('php bin/console doctrine:database:create --env=test');
        shell_exec('php bin/console doctrine:schema:update --force --env=test');
        shell_exec('php bin/console doctrine:fixtures:load -n --env=test');
   }

   // ...vos méthodes de test vont ici...
}
```

Dans cet exemple, la méthode `setUp()` exécute plusieurs commandes pour créer une base de données de test, mettre à jour son schéma pour correspondre à l'état actuel de vos entités Doctrine, et charger des données de test à partir de vos fixtures.

Notez que cette approche suppose que vous avez une base de données distincte pour vos tests, et que cette base de données peut être supprimée et recréée avant chaque test. Si ce n'est pas le cas, vous devrez adapter cette méthode à votre environnement précis.

Types de doubles de test

Les doubles de test sont des objets qui remplacent des objets réels dans un système pour faciliter les tests. Il existe plusieurs types de doubles de test, dont les « *dummies* », les « *stubs* », les « *spies* » et, bien sûr, les « *mocks* ». Chacun a ses propres caractéristiques et usages.

Méthode **Dummies**

Un *dummy* est un objet qui est juste passé, mais jamais utilisé. Il est généralement utilisé pour remplir les listes de paramètres d'une fonction ou d'une méthode. En d'autres termes, vous créez un dummy quand vous avez besoin d'un objet pour passer en tant que paramètre, mais vous ne vous souciez pas de ce que cet objet fait réellement. Dans le contexte des tests unitaires avec Symfony et PHPUnit, vous pouvez créer des dummies en utilisant la méthode `createMock()` ou `getMockBuilder()` de PHPUnit.

Tous droits réservés à STUDI - Reproduction interdite

Exemple

Par exemple, si vous avez une méthode qui prend un objet « *Client* » en paramètre, mais n'utilise pas cet objet « *Client* » de manière significative, alors vous pouvez créer un dummy pour cet objet « *Client* ».

```php
namespace Tests\AppBundle;

use PHPUnit\Framework\TestCase;
use AppBundle\MaClasse;
use AppBundle\Client;

class TestMaClasse extends TestCase
{
   public function testMaMethode()
   {
       $dummyClient = $this->createMock(Client::class); // Création d'un dummy Client
       $maClasse = new MaClasse();

       try {
           $resultat = $maClasse->maMethode($dummyClient);

           // Aucune exception n'a été lancée, le test passe
           $this->assertTrue(true);

           // Ici, on vérifie seulement que $maMethode fonctionne sans erreur.
           // $dummyClient n'est pas utilisé dans l'exécution de $maMethode.

       } catch (\Exception $e) {
           // Une exception a été lancée, le test échoue
           $this->fail('Exception lancée lors de l\'exécution de maMethode : ' . $e->getMessage());
       }
   }
}
```

Ici, dummyClient est un dummy. Il est passé à maMethode simplement pour satisfaire la signature de la méthode. maMethode ne fait rien avec dummyClient dans cet exemple, donc il n'est pas nécessaire que dummyClient ait un comportement spécifique. Sa seule fonction est d'agir comme un remplisseur de place.

Méthode **Les stubs**

Un *stub* est un test double qui a un comportement préprogrammé. Contrairement aux dummies, les stubs sont effectivement utilisés dans l'exécution du test. Ils sont généralement utilisés pour simuler un comportement précis d'une méthode ou d'une fonction, permettant au test de se concentrer sur le code en cours de test et non sur le comportement de ses dépendances.

Exemple

Si par exemple vous testez une classe « *Météo* » qui a une dépendance sur une autre classe « `ServiceMétéo` » qui a une méthode « `obtenirTempérature` », cette méthode fait un appel réseau pour obtenir la température actuelle à partir d'une API météo externe. Lorsque vous testez la classe « `Météo` », vous ne voulez pas que vos tests dépendent de cette API externe. Dans ce cas, vous pouvez créer un stub pour « `ServiceMétéo` » qui renvoie une température fixe, vous permettant de tester la classe « `Météo` » de manière isolée.

```php
namespace Tests\AppBundle;

use PHPUnit\Framework\TestCase;
use AppBundle\Meteo;
use AppBundle\ServiceMeteo;
```


Tous droits réservés à STUDI - Reproduction interdite

```php

class TestMeteo extends TestCase
{
   public function testCalculerVetementPourTemperature()
   {
       $stubServiceMeteo = $this->createMock(ServiceMeteo::class);

       // Configurer le stub pour retourner une température fixe
       $stubServiceMeteo->method('obtenirTemperature')->willReturn(20);

       $meteo = new Meteo($stubServiceMeteo); // Utiliser le stub dans la classe à tester

       $vetement = $meteo->calculerVetementPourTemperature();

       // Ici, vérifier que le bon vêtement est retourné pour une température de 20 degrés
       $this->assertEquals($vetement, 'Vetement approprié pour 20 degrés');
   }
}
```

`$StubServiceMeteo` est un *stub*. Il est utilisé pour remplacer la véritable dépendance sur `ServiceMeteo`, ce qui permet de tester la classe `Meteo` de manière isolée. Le stub est configuré pour renvoyer une température fixe, ce qui rend le test indépendant de l'état actuel de l'API météo externe.

B. Exercice : Quiz

[solution n°1 p.15]

Question 1

Qu'est-ce qu'un test unitaire ?

- ○ Un test d'intégration qui vérifie comment différentes parties du code interagissent
- ○ Un test qui vérifie le comportement d'une unité de code isolée de son environnement
- ○ Un test qui vérifie le comportement global d'une application

Question 2

Qu'est-ce qu'un Dummy dans le contexte des tests unitaires ?

- ○ Un objet qui est passé, mais jamais utilisé
- ○ Un objet avec un comportement préprogrammé
- ○ Un objet qui enregistre des informations sur son utilisation

Question 3

Comment créeriez-vous un dummy avec PHPUnit ?

- ○ Avec createMock()
- ○ Avec createDummy()
- ○ Avec createSpy()

Question 4

Tous droits réservés à STUDI - Reproduction interdite

Qu'est-ce que la méthode setUp() en PHPUnit ?

- ○ Une méthode qui est appelée après chaque test pour nettoyer l'environnement de test
- ○ Une méthode qui est appelée avant chaque test pour préparer l'environnement de test
- ○ Une méthode qui est appelée une seule fois avant tous les tests pour configurer l'environnement de test

Question 5

Quelle est la limitation de l'utilisation des mocks ?

- ○ Les mocks peuvent rendre les tests plus complexes et plus difficiles à comprendre
- ○ Les mocks peuvent rendre les tests plus rapides et plus efficaces
- ○ Les mocks peuvent rendre les tests plus simples et plus faciles à comprendre

III. Approfondissement des doubles de test avec Symfony : Spies et Mocks

A. Approfondissement des doubles de test avec Symfony : Spies et Mocks

Après avoir exploré les concepts de base des doubles de test, nous allons maintenant nous pencher plus en détail sur deux types de doubles de test : les Spies et les Mocks. Ces deux types ont des utilisations spécifiques et apportent un niveau supplémentaire de contrôle et de précision dans la façon dont nous pouvons tester notre code.

Méthode **Les spies**

Un *spy* est un type de test double qui enregistre des informations sur son utilisation. Il peut enregistrer les appels de méthodes, les arguments passés, le nombre de méthodes appelées, etc. Les spies sont généralement utilisés lorsque vous voulez vérifier comment une certaine méthode a été utilisée, plutôt que simplement vérifier son résultat.

Exemple

Supposons que vous ayez une classe « *Logger* » qui a une méthode « *log* » et que vous ayez une autre classe « *Moteur* » qui utilise cette méthode « *log* » pour enregistrer des informations, vous voulez vous assurez que la méthode « *log* » est bien appelée avec les bons arguments lorsque vous utilisez la classe « *Moteur* ».

```php
namespace Tests\AppBundle;

use PHPUnit\Framework\TestCase;
use AppBundle\Moteur;
use AppBundle\Logger;

class TestMoteur extends TestCase
{
   public function testLoggerEstAppeleAvecLesBonsArguments()
   {
       $spyLogger = $this->getMockBuilder(Logger::class)
           ->disableOriginalConstructor()
           ->addMethods(['log'])
           ->getMock();

       $spyLogger->expects($this->once())
           ->method('log')
           ->with($this->equalTo("L'action a été effectuée"));

       $moteur = new Moteur($spyLogger); // Utilisation du spy dans la classe à tester
```


Tous droits réservés à STUDI - Reproduction interdite

```
21
22          $moteur->action();
23      }
24  }
```

Ci-dessus, SpyLogger est un *spy*. Il est utilisé pour remplacer la vraie classe Logger dans le test. Après l'exécution de la méthode action de la classe Moteur, le test vérifie que la méthode log du « *spy* » a été appelée avec le bon argument. Si ce n'est pas le cas, le test échouera.

Complément

Chacun de ces types de tests doubles détient un rôle spécifique dans le cadre des tests unitaires. Pour faire simple :

- Dummy : un dummy est un objet qui est passé, mais jamais utilisé. Il est généralement utilisé pour remplir des listes de paramètres.
- Stub : un Stub est un test double qui a un comportement préprogrammé. Il est utilisé pour simuler une dépendance et son comportement afin que le test puisse se concentrer sur le code à tester.
- Spy : un spy est un test double qui enregistre comment il a été utilisé, comme quels appels de méthode ont été faits ou quels arguments ont été passés. Il est utilisé lorsque vous voulez vérifier comment une certaine méthode a été utilisée.
- Mock : un mock est un test double qui a des attentes prédéfinies sur la façon dont il devrait être utilisé, et qui peut signaler un échec du test si ces attentes ne sont pas satisfaites. Il est utilisé lorsque vous voulez à la fois définir le comportement d'une dépendance (comme un Stub) et vérifier comment cette dépendance est utilisée (comme un spy).

La distinction entre ces types peut être un peu floue, et varie parfois en fonction des outils de test et des langages de programmation. Par exemple, certains outils de test utilisent le terme « *mock* » pour désigner ce qui serait appelé un « *stub* » ou un « *spy* » dans d'autres contextes. C'est pourquoi il est essentiel de bien comprendre les concepts sous-jacents, indépendamment de la terminologie précise utilisée.

Introduction aux méthodes de Mocking

Les mocks sont utilisés en test unitaire pour simuler des objets ou des services qui sont difficiles ou impossibles à implémenter. Par exemple, vous pouvez utiliser un mock pour simuler un service externe qui n'est pas disponible pendant les tests. PHPUnit fournit plusieurs méthodes pour créer et configurer des mocks.

En général, vous créez un mock en appelant la méthode `createMock()` ou `getMockBuilder()` sur votre objet TestCase. Vous pouvez ensuite configurer le mock en appelant des méthodes comme `expects()`, `method()`, `with()`, `will()` et `willReturn()` sur l'objet mock.

Exemple

Voici un exemple de syntaxe générale :

```
1 $mock = $this->getMockBuilder(NomDeLaClasse::class)
2     ->disableOriginalConstructor()
3     ->getMock();
```

Cette syntaxe créée un mock d'une classe en utilisant la méthode getMockBuilder. Ensuite, disableOriginalConstructor est utilisé pour désactiver l'appel au constructeur de la classe d'origine et getMock retourne l'objet mock.

Tous droits réservés à STUDI - Reproduction interdite

Méthode **La méthode 'expects'**

La méthode `expects` permet de définir les attentes sur les appels de méthodes du mock. Elle est généralement utilisée avant d'appeler une méthode du mock.

```
$mock->expects($this->once())
    ->method('nomDeLaMethode')
    ->willReturn('valeurAttendue');
```

Ici, la méthode `nomDeLaMethode` doit être appelée une fois sur le mock, et elle renverra la valeur `'valeurAttendue'`.

Méthode **La méthode 'method'**

La méthode `method()` est utilisée pour préciser quelle méthode du mock vous voulez configurer. Vous passez le nom de la méthode en chaîne de caractères.

Exemple

```
$mock->expects($this->once())
    ->method('myMethod');
```

Ci-dessus, on s'attend à ce que la méthode `myMethod` soit appelée une fois sur le mock.

Méthode **La méthode 'with'**

La méthode `with` spécifie les arguments attendus lors de l'appel d'une méthode.

```
$mock->expects($this->once())
    ->method('nomDeLaMethode')
    ->with($this->equalTo('argumentAttendu'));
```

Dans cet exemple, on définit que `'nomDeLaMethode'` doit être appelée une fois sur le mock avec l'argument `'argumentAttendu'`.

Méthode **Les méthodes 'will' et 'willReturn'**

La méthode will et la méthode willReturn sont deux approches différentes pour préciser la valeur de retour d'une méthode lors de l'utilisation de mocks dans Symfony. Lorsque vous utilisez la méthode will, vous définissez une action à effectuer lors de l'appel de la méthode du mock. Cela peut inclure le renvoi d'une valeur voulu, l'exécution d'une fonction ou la levée d'une exception personnalisée.

Exemple

```
$mock->expects($this->once())
    ->method('nomDeLaMethode')
    ->will($this->returnValue('valeurRetournee'));
```

Ici, l'attente est que la méthode `nomDeLaMethode` soit appelée une fois sur le Mock, et lorsqu'elle est appelée, elle renvoie la valeur 'valeurRetournee'. Vous pouvez personnaliser l'action à effectuer en utilisant d'autres méthodes telles que `willReturnCallback` ou `willThrowException`.

En revanche, la méthode `willReturn` est plus simple et directe, car elle permet d'énoncer directement la valeur de retour attendue pour la méthode du mock.

```
$mock->expects($this->once())
    ->method('nomDeLaMethode')
    ->willReturn('valeurRetournee');
```


Tous droits réservés à STUDI - Reproduction interdite

L'attente est la même que précédemment : la méthode `nomDeLaMethode` doit être appelée une fois sur le Mock, et elle renverra directement la valeur `'valeurRetournee'`.

En résumé, la principale différence entre will et willReturn réside dans la flexibilité offerte par la méthode will pour exécuter des actions personnalisées lors de l'appel d'une méthode du mock, tandis que willReturn se concentre uniquement sur la spécification de la valeur de retour attendue.

B. Exercice : Quiz

[solution n°2 p.16]

Question 1

Qu'est-ce qu'un mock ?

- O Un bogue dans le code
- O Un objet qui simule le comportement d'un véritable objet dans un système
- O Un type spécial de variable en PHP

Question 2

Quand devriez-vous utiliser des mocks dans vos tests ?

- O Lorsque vous voulez tester le comportement d'une unité de code qui interagit avec une ressource extern
- O Lorsque vous voulez tester une petite unité de code qui ne dépend d'aucune autre partie du système
- O Lorsque vous voulez tester le comportement global de votre application

Question 3

À quoi sert method() lors de la configuration d'un mock ?

- O Elle appelle une méthode sur l'objet mock
- O Elle crée une nouvelle méthode sur l'objet mock
- O Elle détermine quelle méthode du mock vous voulez configurer

Question 4

Quelle est la différence entre les méthodes will() et willReturn() lors de la configuration d'un mock ?

- O will() définit une action à effectuer lors de l'appel de la méthode du Mock, tandis que willReturn() spécifie directement la valeur de retour attendue
- O will() appelle une méthode sur l'objet mock, tandis que willReturn() vérifie si une méthode a été appelée
- O Il n'y a pas de différence, les deux méthodes font la même chose

Question 5

Que fait la méthode expects() ?

- O Elle spécifie les attentes sur les appels de méthodes du Mock
- O Elle crée un nouvel objet Mock
- O Elle vérifie si les attentes spécifiées ont été satisfaites

Tous droits réservés à STUDI - Reproduction interdite

IV. Essentiel

Félicitations ! Vous avez fait un grand pas dans votre maîtrise des tests unitaires avec Symfony. Les notions abordées ici étaient nombreuses et diversifiées, et toutes ont leur importance pour le développement de tests efficaces et robustes.

Vous avez été initié aux fondamentaux du testing et des doubles de test dans Symfony. Voici quelques points essentiels à retenir :

- Les tests unitaires sont des procédures utilisées pour valider que des parties individuelles d'un code source fonctionnent comme prévu.
- Les doubles de test sont des remplacements pour les dépendances dans le code qui sont utilisés pendant le testing comme les dummies ou les spies.
- PHPUnit est un framework de test qui permet de créer des tests unitaires pour votre code Un spy est un type de double de test qui enregistre des informations sur son utilisation, telles que les appels de méthodes, les arguments passés et le nombre de méthodes appelées.
- Un mock est un type de double de test qui a des attentes prédéfinies sur la façon dont il devrait être utilisé et qui peut signaler un échec du test si ces attentes ne sont pas satisfaites.

Vous avez également appris comment créer et configurer des mocks en utilisant des méthodes comme createMock(), getMockBuilder(), expects(), method(), with(), will(), et willReturn() dans PHPUnit.

Avec ce cours, vous avez acquis une compréhension solide des tests unitaires et de leur mise en œuvre dans Symfony. Ces connaissances sont essentielles pour développer un code de qualité.

V. Auto évaluation

A. Exercice

Vous êtes développeur pour une entreprise de commerce en ligne et vous êtes chargé de maintenir le module de paiement. Vous utilisez une API externe pour traiter les paiements. Vous devez vous assurer que votre module fonctionne correctement sans faire des requêtes réelles à l'API de paiement.

Question 1

[solution n°3 p.17]

Créez un test unitaire avec un mock de l'API de paiement pour tester la méthode processPayment. Cette méthode doit prendre en paramètre un montant et un identifiant de carte de crédit et renvoyer « *Success* » si le paiement est approuvé ou « *Failure* » si le paiement est refusé.

```php
class PaymentProcessor
{
    public function processPayment($amount, $creditCardId)
    {
        // Code pour traiter le paiement à l'aide d'une API externe
    }
}
```

Au sein de cette même application, vous avez un service TaskService qui gère l'ajout de tâches. Le service TaskService utilise un repository TaskRepository pour interagir avec la base de données.

Question 2

[solution n°4 p.17]

Créez un test qui utilise un test double pour le TaskRepository afin de tester la méthode addTask de TaskService. Cette méthode doit prendre en paramètre un objet Task et renvoyer true si la tâche est ajoutée avec succès, et false sinon.

```php
namespace App\Service;

use App\Entity\Task;
use App\Repository\TaskRepository;

```


Tous droits réservés à STUDI - Reproduction interdite

```php
class TaskService
{
    private $taskRepository;

    public function __construct(TaskRepository $taskRepository)
    {
        $this->taskRepository = $taskRepository;
    }

    public function addTask(Task $task)
    {
        // Code pour ajouter la tâche en utilisant le TaskRepository
    }
}
```

B. Test

Exercice 1 : Quiz

[solution n°5 p.18]

Question 1

Que fait getUserByEmail() ?

- Elle crée un nouvel utilisateur dans la base de données
- Elle recherche un utilisateur dans la base de données
- Elle modifie les informations d'un utilisateur dans la base de données

Question 2

Qu'est-ce qu'une base de données de test ?

- Une copie de la base de données
- La même base de données utilisée en production
- Un type de base de données qui est utilisé uniquement pour les tests unitaires

Question 3

Que signifie « *faire passer tous les tests* » dans le contexte du développement logiciel ?

- S'assurer que tous les tests échouent
- S'assurer que tous les tests réussissent
- S'assurer que tous les tests sont exécutés, qu'ils réussissent ou non

Question 4

Que fait createMock() dans PHPUnit ?

- Elle crée une méthode mock
- Elle crée un objet mock
- Elle crée un objet stub

Question 5

Tous droits réservés à STUDI - Reproduction interdite

Quelle est la principale utilisation d'un stub dans les tests unitaires ?

- ○ Un stub est utilisé pour remplacer les dépendances
- ○ Un stub est utilisé pour enregistrer la façon dont une méthode a été utilisée pendant les tests
- ○ Un stub est utilisé pour remplir les listes de paramètres dans une fonction ou une méthode

Solutions des exercices

Tous droits réservés à STUDI - Reproduction interdite

Exercice p. 7 **Solution n°1**

Question 1

Qu'est-ce qu'un test unitaire ?

- ○ Un test d'intégration qui vérifie comment différentes parties du code interagissent
- ◉ Un test qui vérifie le comportement d'une unité de code isolée de son environnement
- ○ Un test qui vérifie le comportement global d'une application

Un test unitaire est une évaluation ciblée d'une petite unité de code, comme une fonction ou une méthode, afin de vérifier que cette unité fonctionne correctement de manière isolée.

Question 2

Qu'est-ce qu'un Dummy dans le contexte des tests unitaires ?

- ◉ Un objet qui est passé, mais jamais utilisé
- ○ Un objet avec un comportement préprogrammé
- ○ Un objet qui enregistre des informations sur son utilisation

Un Dummy est un objet qui est juste passé, mais jamais utilisé. Il est généralement utilisé pour simuler des paramètres et remplir des signatures de méthodes.

Question 3

Comment créeriez-vous un dummy avec PHPUnit ?

- ◉ Avec createMock()
- ○ Avec createDummy()
- ○ Avec createSpy()

Dans le contexte des tests unitaires avec Symfony et PHPUnit, vous pouvez créer des Dummies en utilisant la méthode createMock() ou getMockBuilder() de PHPUnit.

Question 4

Qu'est-ce que la méthode setUp() en PHPUnit ?

- ○ Une méthode qui est appelée après chaque test pour nettoyer l'environnement de test
- ◉ Une méthode qui est appelée avant chaque test pour préparer l'environnement de test
- ○ Une méthode qui est appelée une seule fois avant tous les tests pour configurer l'environnement de test

La méthode setUp() en PHPUnit est appelée automatiquement avant chaque test. Vous pouvez utiliser cette méthode pour préparer et configurer votre environnement de test.

Question 5

Quelle est la limitation de l'utilisation des mocks ?

Tous droits réservés à STUDI - Reproduction interdite

- [x] Les mocks peuvent rendre les tests plus complexes et plus difficiles à comprendre
- [ ] Les mocks peuvent rendre les tests plus rapides et plus efficaces
- [ ] Les mocks peuvent rendre les tests plus simples et plus faciles à comprendre

Bien que les mocks soient utiles, ils peuvent rendre les tests plus complexes et plus difficiles à comprendre, surtout si vous utilisez beaucoup de mocks ou des mocks qui ont des comportements très élaborés.

Exercice p. 11 **Solution n°2**

Question 1

Qu'est-ce qu'un mock ?

- [ ] Un bogue dans le code
- [x] Un objet qui simule le comportement d'un véritable objet dans un système
- [ ] Un type spécial de variable en PHP

Un mock est une simulation programmée d'un objet réel qui reproduit son comportement de manière contrôlée. Les mocks sont utilisés pour isoler le code à tester et maîtriser le comportement des dépendances.

Question 2

Quand devriez-vous utiliser des mocks dans vos tests ?

- [x] Lorsque vous voulez tester le comportement d'une unité de code qui interagit avec une ressource extern
- [ ] Lorsque vous voulez tester une petite unité de code qui ne dépend d'aucune autre partie du système
- [ ] Lorsque vous voulez tester le comportement global de votre application

Les mocks sont particulièrement utiles lorsque vous voulez isoler une partie du code pour la tester indépendamment de ses interactions avec d'autres éléments du système, en particulier lors de ces interactions sont complexes, lentes ou imprévisibles.

Question 3

À quoi sert method() lors de la configuration d'un mock ?

- [ ] Elle appelle une méthode sur l'objet mock
- [ ] Elle crée une nouvelle méthode sur l'objet mock
- [x] Elle détermine quelle méthode du mock vous voulez configurer

La méthode method() est utilisée pour indiquer quelle méthode du mock vous voulez configurer. Vous passez le nom de la méthode en chaîne de caractères.

Question 4

Quelle est la différence entre les méthodes will() et willReturn() lors de la configuration d'un mock ?

- [x] will() définit une action à effectuer lors de l'appel de la méthode du Mock, tandis que willReturn() spécifie directement la valeur de retour attendue
- [ ] will() appelle une méthode sur l'objet mock, tandis que willReturn() vérifie si une méthode a été appelée
- [ ] Il n'y a pas de différence, les deux méthodes font la même chose

Tous droits réservés à STUDI - Reproduction interdite

La méthode will() définit une action voulue à effectuer lors de l'appel de la méthode du Mock, tandis que willReturn() vous permet de préciser directement la valeur de retour attendue.

Question 5

Que fait la méthode expects() ?

- (●) Elle spécifie les attentes sur les appels de méthodes du Mock
- ( ) Elle crée un nouvel objet Mock
- ( ) Elle vérifie si les attentes spécifiées ont été satisfaites

La méthode expects() est utilisée pour préciser les attentes concernant les appels de méthodes du Mock. Par exemple, on peut s'attendre à ce qu'une méthode soit appelée une certaine fois ou jamais.

p. 12 Solution n°3

Lorsque nous testons la méthode processPayment, au lieu d'effectuer un véritable appel à l'API, nous utilisons notre mock qui retourne directement « *Success* ». De cette façon, le test s'effectue sans dépendre de la vraie API de paiement.

```php
class PaymentProcessorTest extends PHPUnit\Framework\TestCase
{
    public function testProcessPayment()
    {
        $mockApi = $this->getMockBuilder('PaymentApi')
            ->setMethods(['processPayment'])
            ->getMock();

        $mockApi->method('processPayment')->willReturn('Success');

        $paymentProcessor = new PaymentProcessor($mockApi);
        $result = $paymentProcessor->processPayment(100, '1234');

        $this->assertEquals('Success', $result);
    }
}
```

p. 12 Solution n°4

Ici, nous testons la méthode addTask avec un double de test. Ce double est configuré pour retourner true quand la méthode save est appelée. Ainsi, nous pouvons tester addTask sans avoir besoin d'une vraie base de données.

```php
namespace App\Tests\Service;

use App\Service\TaskService;
use App\Entity\Task;
use App\Repository\TaskRepository;
use PHPUnit\Framework\TestCase;

class TaskServiceTest extends TestCase
{
    public function testAddTask()
    {
        $taskRepository = $this->createMock(TaskRepository::class);

```

Tous droits réservés à STUDI - Reproduction interdite

```
14          $taskRepository->expects($this->once())
15              ->method('save')
16              ->willReturn(true);
17
18          $taskService = new TaskService($taskRepository);
19
20          $task = new Task();
21          $task->setName('Test Task');
22
23          $result = $taskService->addTask($task);
24
25          $this->assertTrue($result);
26      }
27  }
```

Exercice p. 13 **Solution n°5**

Question 1

Que fait getUserByEmail() ?

- ○ Elle crée un nouvel utilisateur dans la base de données
- ◉ Elle recherche un utilisateur dans la base de données
- ○ Elle modifie les informations d'un utilisateur dans la base de données

La méthode getUserByEmail() est utilisée pour rechercher un utilisateur dans la base de données de par son adresse e-mail.

Question 2

Qu'est-ce qu'une base de données de test ?

- ◉ Une copie de la base de données
- ○ La même base de données utilisée en production
- ○ Un type de base de données qui est utilisé uniquement pour les tests unitaires

Une Base de données de test est une BDD avec des fixtures (des données simulées) pour correspondre à une BDD de prod, mais sans en utiliser les données réelles. Cela évite de perturber les données de production pendant les tests.

Question 3

Que signifie « *faire passer tous les tests* » dans le contexte du développement logiciel ?

- ○ S'assurer que tous les tests échouent
- ◉ S'assurer que tous les tests réussissent
- ○ S'assurer que tous les tests sont exécutés, qu'ils réussissent ou non

« *Faire passer tous les tests* » signifie que tous les tests réussissent. C'est l'objectif lors de l'exécution des tests, car cela indique que votre code fonctionne comme prévu.

Question 4

Que fait createMock() dans PHPUnit ?

Tous droits réservés à STUDI - Reproduction interdite

- ○ Elle crée une méthode mock
- ◉ Elle crée un objet mock
- ○ Elle crée un objet stub

La méthode createMock() est utilisée pour créer un objet mock qui peut simuler un objet ou un service spécifique.

Question 5

Quelle est la principale utilisation d'un stub dans les tests unitaires ?

- ◉ Un stub est utilisé pour remplacer les dépendances
- ○ Un stub est utilisé pour enregistrer la façon dont une méthode a été utilisée pendant les tests
- ○ Un stub est utilisé pour remplir les listes de paramètres dans une fonction ou une méthode

Un stub est un test double qui a un comportement préprogrammé. Il remplace les dépendances pour l'exécution de tests.

Tous droits réservés à STUDI - Reproduction interdite